SECURITY · CREDENTIALS

CRM Sync — Key Management Lifecycle

Dev → Stage → Prod key management as one lifecycle: the **Interactive Key Ceremony** (agent-safe credential operations), rotation cadence, RACI ownership, self-service tenant keys, and server-side session revocation.

Version: **1.6**

Date: **2026-10-23 (v1.5: 2026-07-03; v1.4: 2026-06-22; v1.2: 2026-06-15; v1.1: 2026-06-11; v1.0: 2026-05-26)**

Scope: **Dev → Stage → Prod key management, consulting team workflow, stakeholder publish, per-market site tokens, Interactive Key Ceremony (agent-safe credential operations), .env/CLI-as-AI-exposure-surface threat model, organizational RACI + tech→business ownership, regulated-industry glossary, GraphQL/server-side migration significance, self-service tenant key management (app owners, §13), server-side session revocation, email authentication — SPF, DKIM, DMARC on Cloudflare DNS (§8.4)**

Companion: **KEY-CEREMONY-LOOP-REVIEW.md — recurring, non-custodial automation checklist that continuously attests this lifecycle (the QA/Compliance *verify* lane as a /loop).**

---

0. Executive Summary (Challenge → Solution)

Each pillar of this lifecycle stated as one plain challenge/solution sentence, with the supporting detail bulleted beneath. Section references point to the full treatment.

Agent-safe key ceremony (§9). *Challenge:* an AI/automation agent is a superb operator but a liability as a secret-holder — once it can mint or read a key, that secret's reach includes its context window, the transcript, and every log it touches. *Solution:* run every privileged credential op as a two-role ceremony where a **human executes** and the **agent only prepares, verifies, and records**.

- A permission classifier **blocks** the agent from running credential-minting commands.

- Secrets are injected by reference (`$(cat ~/.crm-*-key)`) — never inline, printed, or committed.
- Read-back is **masked** (`564bbe…9298`) or a `(set)` flag — the value never returns.
- Rotation is **additive-only** (root key never replaced), so a botched ceremony can't lock anyone out.
- Every ceremony files a **fingerprint-only** audit record (`sha256[:8]`), safe to publish.

`.env` + CLI as an AI exposure surface (§9.6). *Challenge:* the everyday way to manage secrets — a `.env` file plus `wrangler` / `curl` at the terminal — is exactly what leaks to an agent sharing the machine. *Solution:* treat `.env` and the CLI as **hostile-to-AI by default** and route every secret around the agent.

- Privileged secrets live in **write-only Cloudflare secrets / KV**, never a checked-out `.env`.
- The **human** runs the interactive command; the agent is blocked and hands it over.
- Verification is by **side-effect** (a `200`, an old key now `401`), never by echoing the value.
- Rotation happens in the **config layer** (KV beats `env`) — no `.env` edit, no redeploy.

Organizational roles & RACI (§9.7, §12). *Challenge:* a bare two-person ceremony doesn't say who does what once a real org is involved. *Solution:* map functional roles onto the ceremony with a **RACI matrix** that has exactly **one Accountable** per activity.

- **Deployment Officer** executes the write (= the Security Human).
- **PMO** schedules; **QA/Release Manager** verifies stage→prod; **Compliance Officer** attests cadence.

- **DPO** approves anything touching the PII plane — but is **Consulted, never Accountable**.
- The **Agent never appears as Responsible or Accountable** for a privileged write.

Formal definitions for regulated industry (§10). *Challenge:* auditors won't accept colloquial terms — controls must map to named standards. *Solution:* a **glossary** giving each term its normative definition and citation.

- **Dual control** & **split knowledge** → PCI-DSS; **SoD** & **least privilege** → NIST SP 800-53.
- **Crypto Officer** → FIPS 140-3; **cryptoperiod** & **key rotation** → NIST SP 800-57.
- **DPO** & **security of processing** → GDPR Art. 37–39 / Art. 32.
- Our "blast radius" is formalized as **scope of compromise**; "fingerprint" as a truncated key hash.

GraphQL / server-side migration significance (§11). *Challenge:* Shopify→GraphQL and Google→GA4-server-side **concentrate** credential power into fewer, high-privilege, server-resident secrets — just as AI agents do more of the integration work. *Solution:* make **least-privilege scoping + rotation + the ceremony** load-bearing controls at the worker boundary.

- One GraphQL endpoint = one token = a **whole-schema** scope of compromise.
- GA4 server-side measurement introduces a **real** `api_secret` where client-side had none.
- Platform-native **token expiry/rotation** now aligns with the §8.1 cadence.
- Agents that **codegen GraphQL** must stay non-custodial (§9.1).

Ownership, tech → business (§12). *Challenge:* without a single owner, accountability diffuses and the control silently rots. *Solution:* name **one Accountable per activity**, escalating from execution on the technical floor to **business risk ownership**.

- **Deployment Officer / Engineering Lead** carry technical Responsibility.
- **CISO** (or Head of Security / VP Eng) is **Accountable for residual risk** — a business function.
- **DPO** is Consulted; the **Agent** is never A or R for a write.
- Two lines must never collapse: **executor ≠ attester**, and **Accountable ≠ the doer**.

Self-service tenant key management (§13). *Challenge:* every key reset routed through the platform operator makes the operator a bottleneck — and a custodian — for credentials that rightfully belong to each app owner. *Solution:* the purchased product **includes the key system**: owners run their own lifecycle through an entitlement-gated wizard; the platform demonstrates it but never holds the key.

- One dedicated key per store; ownership proven by **purchase-granted entitlement**, never self-assigned.
- Rotation is **mint-before-revoke** — a mid-flight failure can never leave the owner keyless.
- The key is returned **exactly once** at mint; list views show fingerprints only.
- Every rotate/revoke lands in a per-store, **fingerprint-only append-only audit ledger**.
- Login sessions are **revoked server-side on logout** (denylist), so a cached session cannot outlive its logout.

---

1. Environments

ENVIRONMENT	WORKER NAME	DOMAIN	SHOPIFY STORE	PURPOSE
Dev	local (`wrangler dev`)	`localhost:8787`	—	Local development, no real secrets
Stage	`cf-worker-crm-sync`	`crm.story-story.ai`	`hx-stage.myshopify.com`	Integration testing, client demos
Prod	`cf-worker-crm-sync-prod` *(or client's worker)*	`crm.{client-domain}`	`{client}.myshopify.com`	Live customer traffic

2. Key Types

KEY	STORAGE	SCOPE	ROTATABLE VIA API	ENV-SPECIFIC
Root ADMIN_KEY	Cloudflare secret	Worker-wide	No (CLI only)	Yes — different per env
Rotatable admin key	KV `admin_key:rotatable`	Worker-wide	Yes (`POST /admin/rotate-key`)	Yes — lives in env's KV
Tenant token (`crm_t_*`)	KV `tenant_token:{token}`	Shop-scoped	Provision/revoke via API	Yes — lives in env's KV
JWT_SECRET	Cloudflare secret	Session signing	No (CLI only)	Yes — different per env
GOOGLE_CLIENT_SECRET	Cloudflare secret	OAuth	No (CLI only)	Yes — per Google project
SHOPIFY_ADMIN_TOKEN	KV (per-tenant config)	Shop-scoped	Auto-rotated (expiring tokens)	Yes — per Shopify app
XANO_API_KEY	Cloudflare secret or KV	Database access	No	Yes — per Xano workspace
RESEND_API_KEY	Cloudflare secret	Transactional email	No	Yes — per Resend account
GA4_API_SECRET	Cloudflare secret or KV	Analytics	No	Yes — per GA4 property
WEBFLOW_CMS_TOKEN	KV (auto-set by OAuth)	CMS writes	Auto-set by Webflow OAuth	Yes — per Webflow site
Market site token	KV `market_cms_token:{region}`	One market's Webflow site	Re-set via API (write-only)	Yes — one per market clone

3. Dev Environment

Who: Engineering team (Persona A)

```
# Start local dev
cd workers/crm-sync
wrangler dev --config wrangler.toml
```

Key rules:

- Use `.dev.vars` file for local secrets (never committed)
- Generate throwaway keys: `openssl rand -hex 16`
- No real Shopify/Google/Xano credentials — use test accounts
- KV is local (miniflare) — tenant tokens and rotatable keys are ephemeral

`.dev.vars` example:

```
ADMIN_KEY=dev-only-throwaway-key-1234567890
JWT_SECRET=dev-jwt-secret-not-for-production
XANO_API_KEY=xano-dev-key
GOOGLE_CLIENT_SECRET=google-dev-secret
RESEND_API_KEY=re_test_xxxxx
```

Extension dev:

```
cd extensions/crm-auth
npm run dev    # webflow extension serve 1338 + tsc --watch
```

4. Stage Environment

Who: Engineering + consulting team **Worker:** `cf-worker-crm-sync` on `crm.story-story.ai` **Store:** `hx-stage.myshopify.com`

4.1 Initial Stage Setup

```
# Set stage secrets (one-time)
cd workers/crm-sync
openssl rand -hex 24 | wrangler secret put ADMIN_KEY --config wrangler.toml
openssl rand -hex 24 | wrangler secret put JWT_SECRET --config wrangler.toml
wrangler secret put GOOGLE_CLIENT_SECRET --config wrangler.toml   # from Google Console
wrangler secret put XANO_API_KEY --config wrangler.toml           # from Xano stage workspace
```

4.2 Stage Key Inventory

KEY	VALUE SOURCE	WHO HOLDS IT
Root ADMIN_KEY	Generated at deploy	Engineering lead only
Rotatable key	Created via API for consulting team	Consulting team lead
Tenant token (`crm_t_*`)	Provisioned per demo client	Consulting team member
Google OAuth	`story-story` Google project	Engineering lead
Shopify Admin	Auto-set by OAuth install on hx-stage	Worker (KV)

4.3 Consulting Team Stage Access

```
# Engineering lead creates rotatable key for consulting team
curl -X POST https://crm.story-story.ai/admin/rotate-key \
  -H "Authorization: Bearer $ROOT_ADMIN_KEY" \
  -H "Content-Type: application/json" \
  -d '{"key":"consulting-stage-key-minimum-20chars"}'

# Consulting team uses this key for all stage admin operations
# They CANNOT access root key or CLI — only API-level operations
```

Consulting team can:

- Access `/setup`, `/settings`, `/onboarding` with their key
- Provision tenant tokens for demo clients
- Read/write config for any shop on the stage worker
- Rotate their own key if compromised

Consulting team cannot:

- Access Cloudflare dashboard or CLI
- Change root ADMIN_KEY
- Modify worker code or deploy
- Access other Cloudflare secrets (JWT_SECRET, etc.)

> **Verified in practice (2026-06-11):** the full delegation lifecycle — root-authenticated create, masked status check (`GET /admin/rotate-key` returns a `key_preview` only, never the key), delegate authentication on admin endpoints, self-rotation, and root-only revocation (`DELETE /admin/rotate-key`) — is exercised and confirmed on stage. Rotating or revoking the delegate key never invalidates the root key or tenant tokens.

4.4 Market Site Tokens (multi-site localization)

Each localized market (e.g. a per-country Webflow site clone) authenticates with its **own site-scoped token**, because Webflow site tokens cannot cross sites:

```
# Store a market's site token (write-only field; created in that site's
# Webflow settings → Apps & Integrations → API access)
curl -X POST https://crm.story-story.ai/admin/markets \
  -H "Authorization: Bearer $ADMIN_OR_DELEGATE_KEY" \
  -H "Content-Type: application/json" \
  -d '{"region":"ca","cms_token":"<site token>"}'
```

Security properties:

- **Write-only**: no endpoint ever returns the token. Reads show a `"(set)"` flag only.
- **Stored apart from config**: KV `market_cms_token:{region}`, never inside the market config object that `GET /admin/markets` returns.
- **Scoped blast radius**: a leaked market token exposes exactly one site — not the primary site, not the worker, not other markets.
- **Content gating**: public localized-export endpoints serve only `reviewed`/`published` translation overrides; machine-translation drafts cannot leak through any public surface.

---

5. Prod Environment

Who: Depends on stakeholder tier

5.1 Persona A (App Creator) — Prod Deploy

```
# Create production worker (separate from stage)
cd workers/crm-sync
cp wrangler.toml wrangler.prod.toml
# Edit wrangler.prod.toml: name = "cf-worker-crm-sync-prod", new KV namespace ID

# Set production secrets
openssl rand -hex 32 | wrangler secret put ADMIN_KEY --config wrangler.prod.toml
openssl rand -hex 32 | wrangler secret put JWT_SECRET --config wrangler.prod.toml
wrangler secret put GOOGLE_CLIENT_SECRET --config wrangler.prod.toml
wrangler secret put XANO_API_KEY --config wrangler.prod.toml
wrangler secret put RESEND_API_KEY --config wrangler.prod.toml

# Deploy
wrangler deploy --config wrangler.prod.toml

# Set up custom domain
# (via Cloudflare Workers Custom Domains API or dashboard)
```

5.2 Persona B (Shared) — Prod Onboarding

```
# A provisions tenant token for B's store
curl -X POST https://crm.{prod-domain}/admin/provision-token \
  -H "Authorization: Bearer $PROD_ADMIN_KEY" \
  -H "Content-Type: application/json" \
  -d '{
    "shop": "client-store.myshopify.com",
    "label": "client-name",
    "scopes": ["config:read", "config:write"]
  }'
# → Returns: crm_t_xxxx

# B receives ONLY:
#   1. Tenant token (crm_t_xxxx)
#   2. Worker URL (https://crm.{prod-domain})
#   3. Extension auto-fills the URL
# B never sees: root key, Cloudflare secrets, KV internals
```

5.3 Persona C (Private Worker) — Prod Handoff

```
# C deploys their own worker from the source code license

cd workers/crm-sync

wrangler deploy --config wrangler.toml   # their own Cloudflare account


# C sets their own root key

openssl rand -hex 32 | wrangler secret put ADMIN_KEY --config wrangler.toml


# C creates a rotatable key for day-to-day operations

curl -X POST https://their-worker.their-domain.com/admin/rotate-key \

  -H "Authorization: Bearer $THEIR_ROOT_KEY" \

  -H "Content-Type: application/json" -d '{}'

# → Returns auto-generated rotatable key


# C provisions tenant tokens for each of their Shopify stores

curl -X POST https://their-worker.their-domain.com/admin/provision-token \

  -H "Authorization: Bearer $ROTATABLE_KEY" \

  -H "Content-Type: application/json" \

  -d '{"shop":"store-1.myshopify.com","label":"store-1"}'


curl -X POST https://their-worker.their-domain.com/admin/provision-token \

  -H "Authorization: Bearer $ROTATABLE_KEY" \

  -H "Content-Type: application/json" \

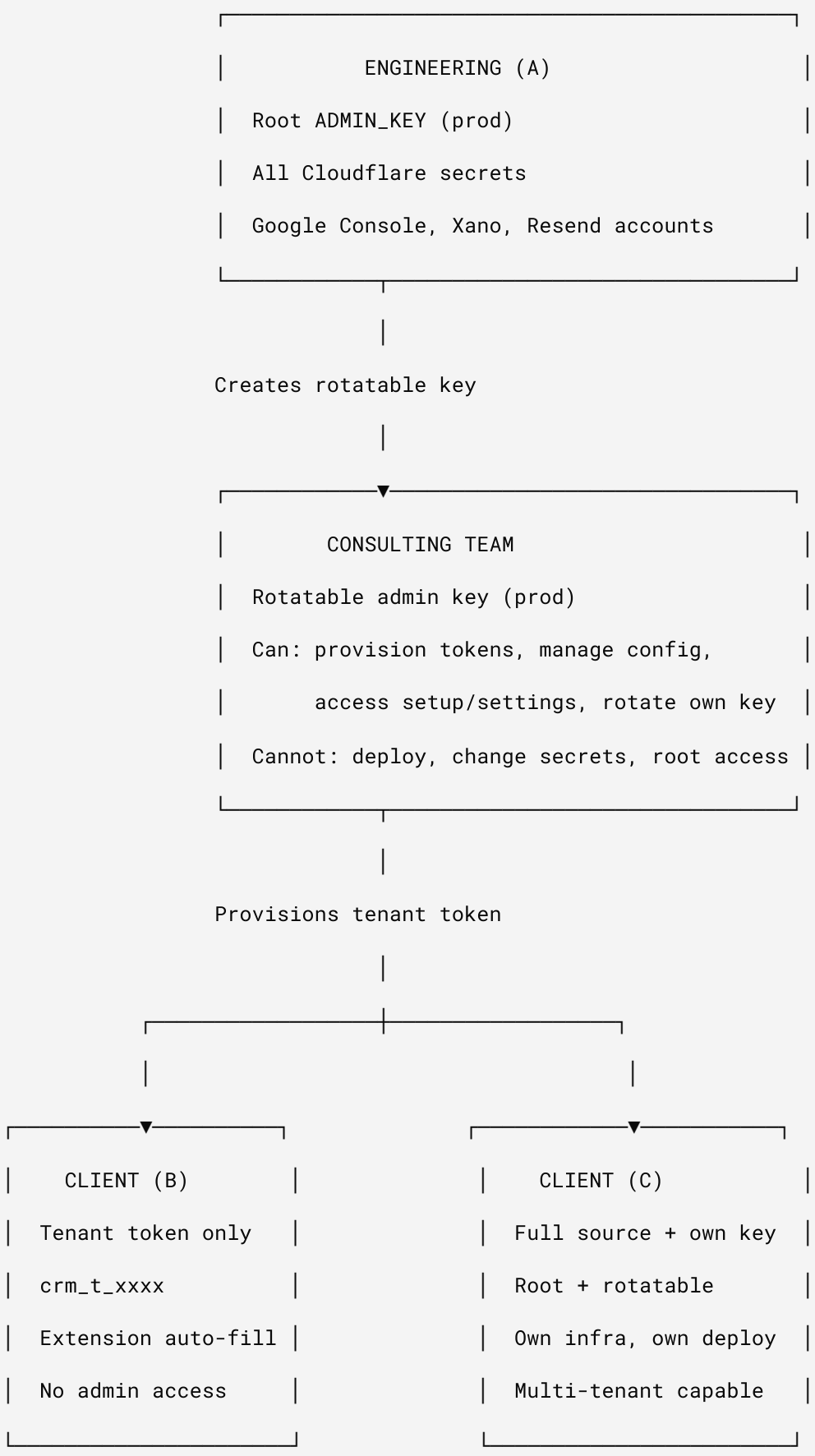
  -d '{"shop":"store-2.myshopify.com","label":"store-2"}'
```

6. Consulting Team → Live Deploy Workflow

6.1 Pre-Deploy Checklist

#	TASK	ENVIRONMENT	WHO
1	Verify all integrations on stage	Stage	Consulting team
2	Document client's required OAuth scopes	Stage	Consulting team
3	Request prod deploy from engineering	—	Consulting team → A
4	Create prod worker or provision on shared	Prod	A (engineering)
5	Set prod secrets	Prod	A (engineering)
6	Create consulting rotatable key for prod	Prod	A → consulting team
7	Provision client tenant token	Prod	Consulting team
8	Client installs Shopify app (OAuth flow)	Prod	Client (B)
9	Client connects Google OAuth in extension	Prod	Client (B)
10	Verify end-to-end auth flow	Prod	Consulting team
11	Rotate consulting key (remove team access)	Prod	A or consulting lead

6.2 Key Handoff Matrix



6.3 Post-Launch Key Rotation

After consulting team completes handoff:

```
# Option 1: Rotate consulting key (new key, team retains support access)
curl -X POST https://crm.{prod-domain}/admin/rotate-key \
  -H "Authorization: Bearer $CONSULTING_KEY" \
  -H "Content-Type: application/json" -d '{}'
# Old consulting key instantly invalid, new key for ongoing support

# Option 2: Revoke consulting key entirely (root-only)
curl -X DELETE https://crm.{prod-domain}/admin/rotate-key \
  -H "Authorization: Bearer $ROOT_ADMIN_KEY"
# Only root key valid — consulting team fully cut off

# Option 3: Issue new consulting key with different holder
curl -X POST https://crm.{prod-domain}/admin/rotate-key \
  -H "Authorization: Bearer $ROOT_ADMIN_KEY" \
  -H "Content-Type: application/json" \
  -d '{"key":"new-support-team-key-at-least-20"}'
```

---

7. Stakeholder Publish — Key Inventory

7.1 What Each Stakeholder Receives at Publish

CREDENTIAL	A (CREATOR)	B (SHARED)	C (PRIVATE)
Root ADMIN_KEY	✓ Generated + held	✕ Never	✓ They generate
Rotatable key	✓ Optional	✕ Never	✓ Self-service
Tenant token	N/A (is the admin)	✓ `crm_t_*`	✓ For sub-tenants
Worker URL	Own domain	Provided (auto-fill)	Own domain
Extension URL	`/extension/` on own worker	Same (A's worker)	`/extension/` on own worker
Google Client ID	Own project	Shared (A's)	Own project
Google Client Secret	Own project	✕ Never	Own project
Shopify Admin Token	Auto via OAuth	Auto via OAuth	Auto via OAuth
Xano API Key	Own workspace	✕ Never	Own workspace
Cloudflare dashboard	✓ Full access	✕ Never	✓ Own account
Source code	✓ Full repo	✕ Never	✓ Licensed copy

7.2 Publish Ceremony (Per Stakeholder)

Stakeholder B publish:

1. A provisions `crm_t_*` token scoped to B's shop
2. A sends B: tenant token + worker URL
3. B installs Webflow extension → auto-fills worker URL
4. B enters tenant token in Config tab
5. B installs Shopify app (OAuth flow auto-provisions admin token)

6. B connects Google login via extension Auth tab
7. Done — B operates entirely through extension UI

Stakeholder C publish:

1. A delivers licensed source code repo
2. C deploys to their Cloudflare account (`wrangler deploy`)
3. C sets root ADMIN_KEY via CLI
4. C creates rotatable key via API (for team operations)
5. C sets up their own: Google project, Xano workspace, Resend account
6. C registers their Shopify app in Partners
7. C updates Webflow extension URL to their worker domain
8. C provisions tenant tokens per Shopify store
9. C sets `plan: "private"` in config
0. Done — C is fully independent, A has no access

---

8. Security Policies

8.1 Key Rotation Schedule

KEY	ROTATION FREQUENCY	WHO ROTATES	METHOD
Root ADMIN_KEY	Annually or on personnel change	A (engineering)	`wrangler secret put`
Rotatable key	Quarterly or on team change	Key holder	`POST /admin/rotate-key`
Tenant tokens	On client offboarding or breach	A or consulting team	`POST /admin/revoke-token`
JWT_SECRET	Annually (invalidates all sessions)	A (engineering)	`wrangler secret put`
Google Client Secret	On suspected compromise	A (engineering)	Google Console + `wrangler secret put`
Shopify Admin Token	Auto-rotated (expiring tokens)	Worker (cron)	Automatic
DKIM signing keys	Yearly, or on any exposure at the sending provider	A (engineering)	New selector in Cloudflare DNS, switch, remove the old one (§8.4)

8.2 Incident Response — Key Compromise

SCENARIO	IMMEDIATE ACTION	RECOVERY
Root key leaked	`wrangler secret put ADMIN_KEY` with new value	Rotate all dependent keys, audit KV access logs
Rotatable key leaked	`POST /admin/rotate-key` (self-rotation)	Audit what was accessed, notify affected clients
Tenant token leaked	`POST /admin/revoke-token` + provision new	Notify affected client, review shop config for tampering
JWT_SECRET leaked	`wrangler secret put JWT_SECRET`	All user sessions invalidated — users must re-login
Consulting key leaked	`DELETE /admin/rotate-key` (root revoke)	Audit operations during exposure window

8.3 Environment Isolation Rules

- **Never** use prod keys in stage or dev
- **Never** share root keys via email, Slack, or any unencrypted channel
- **Never** commit secrets to git (use `.dev.vars` locally, `wrangler secret put` remotely)
- Stage and prod use **separate** KV namespaces — no cross-contamination
- Consulting team gets **rotatable key only** — never root, never CLI access

8.4 Email Authentication — SPF, DKIM and DMARC on Cloudflare DNS

Why this belongs in a key document. DKIM is a signing key. The sending provider holds the private half; the public half is published in DNS, the same pattern as the worker's JWKS. SPF says which servers may send for the domain, and DMARC tells receiving mail servers what to do when a message fails both checks. Without them, anyone can send a login link, a password reset or an invoice that appears to come from the domain — the cheapest phishing there is.

Where it lives. The `crm-sync.dev` zone is on Cloudflare DNS. Inbound mail goes to Google Workspace; transactional mail (login and demo emails) goes out through Resend on the `send.` subdomain, so the two never share a return path. Cloudflare Email Routing is disabled for this zone. Mail records are DNS only — MX and TXT records are never proxied.

RECORD	NAME	VALUE (PUBLIC DNS)	PURPOSE
MX	`crm-sync.dev`	`smtp.google.com` (priority 1)	Inbound mail to Google Workspace
SPF (TXT)	`crm-sync.dev`	`v=spf1 include:_spf.google.com ~all`	Only Google may send as the apex
SPF (TXT)	`send.crm-sync.dev`	`v=spf1 include:amazonses.com ~all`	Resend's sending servers, for the `send.` return path
MX	`send.crm-sync.dev`	`feedback-smtp.us-east-1.amazonses.com` (10)	Bounces and complaints back to Resend
DKIM (TXT)	`google._domainkey`	RSA 2048-bit public key	Signs mail sent from Google Workspace
DKIM (TXT)	`resend._domainkey`	RSA 1024-bit public key	Signs mail sent through Resend
DMARC (TXT)	`_dmarc.crm-sync.dev`	`v=DMARC1; p=none; sp=none; adkim=r; aspf=r`	Monitoring only — failures are reported, not blocked

Check them from any machine — public records, no credentials:

```
dig +short TXT crm-sync.dev

dig +short TXT _dmarc.crm-sync.dev

dig +short TXT send.crm-sync.dev

dig +short TXT google._domainkey.crm-sync.dev

dig +short TXT resend._domainkey.crm-sync.dev
```

Who may change them. Mail DNS is a credential surface: whoever can edit the zone can authorise a new sender. Edits go through the Cloudflare dashboard by A (engineering), or through an API token scoped to **Zone → DNS → Edit on this zone only** — never a global API key, and never a token handed to an agent. Each change is recorded like any other key operation (§9.4): who, which record, old and new value, why.

Recommendations, in order:

1. **Collect DMARC reports.** Add a `rua=` address to the DMARC record — Cloudflare's DMARC Management can receive and summarise them — so failures are visible before policy tightens.
2. **Tighten DMARC once reports are clean:** `p=none` → `p=quarantine` → `p=reject`, with `sp=` following for subdomains. `p=none` is a starting point, not a control.
3. **Move the Resend DKIM key to 2048-bit** if Resend offers it for the domain. A 1024-bit RSA key gives about 80 bits of security, below the 112-bit minimum NIST has required for new signatures since the end of 2013 (SP 800-131A).
4. **Harden SPF to** `-all` once every legitimate sender is listed, and keep each record under SPF's limit of ten DNS lookups.
5. **Rotate DKIM by selector, never in place.** Publish the new selector, switch the provider to it, wait at least a week for mail in transit, then delete the old record. Rotate yearly and whenever the provider reports a key exposure.
6. **One provider key per environment that sends mail.** Dev uses a test key in `.dev.vars`; any environment that sends real mail holds its own key, made in its own ceremony. A key that can send from Prod's domain never sits in Stage.
7. **Send only from verified subdomains.** A new sender gets its own subdomain, its own SPF and DKIM, and a DMARC review — never a new `include:` on the apex added in a hurry.

---

9. Interactive Key Ceremony (Agent-Safe Credential Operations)

> **Primary feature.** Every privileged credential operation in this stack — mint, rotate, revoke, set — runs as a **two-role ceremony**: a named **Security Human** *executes* the privileged command; an **Agent** (AI/automation, e.g. Claude Code) *prepares, verifies, and records* but **never mints, holds, or transports a secret.** This is the normative procedure behind `FUNCTIONAL-SPEC.md` §13 FR-HANDOFF-02.

9.0 Heritage — an established discipline, applied to a new threat

"Key ceremony" is **not** a term invented for this project. It is a long-standing practice in cryptography and high-assurance operations:

- **Root CA key generation** — Certificate Authorities create root signing keys in a scripted, witnessed, audited ceremony, under *split knowledge* and *dual control*, so no single person ever holds the whole key.
- **DNSSEC Root KSK ceremony** — ICANN runs a quarterly, publicly attested ceremony to sign the DNS root, with named role-holders (Crypto Officers, Internal Witnesses, a Ceremony Administrator).
- **HSM / payment key ceremonies** — banks and PCI-DSS environments load master keys into hardware security modules under dual control, with key components held by separate custodians.

The classic ceremony defends against a **malicious or careless human insider**. Our contribution is to extend the *same* primitives — split roles, dual control, verify-don't-view, fingerprint-only audit — to a **new untrusted party: the AI/automation agent**. The "Interactive Key Ceremony" treats a capable coding/ops agent the way a CA treats any single custodian: useful for everything *around* the secret, but never the holder of it. Framing it this way is deliberate — this is a known discipline applied to the agentic-operations threat surface, not a bespoke coinage.

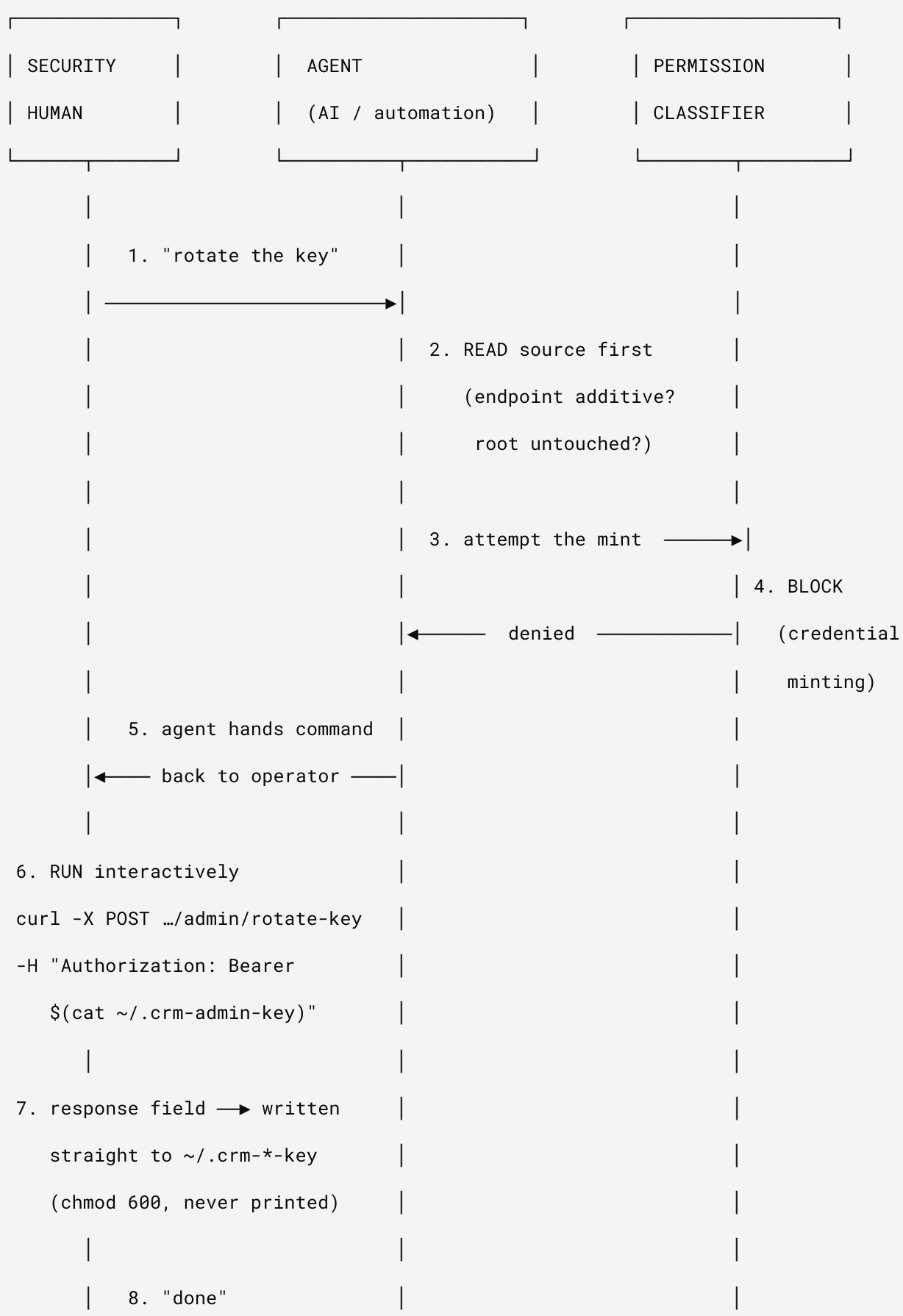
9.1 Why this exists

A capable coding/ops agent is a force multiplier for everything *except* secret custody. The moment an agent can mint or read a production credential, that secret's blast radius includes the model's context window, the transcript, and any log it touches. The ceremony removes the agent from the secret path by construction — not by policy reminder — so the agent stays maximally useful (it writes the runbook, runs every verification probe, files the audit record) while the secret never enters a surface it shouldn't.

Three properties make it safe **by construction**, not by good behaviour:

1. **The agent never executes the privileged write.** A permission classifier blocks credential-minting commands; only the Security Human's interactive invocation runs it. A blocked mint is *handed to the operator* — never worked around.
2. **The secret never enters the transcript.** The new key's value is written straight to disk (`~/.crm-*-key`, `chmod 600`) or piped directly into `wrangler secret put`. Read-back surfaces return a **masked preview** (`564bbe…9298`) or a `"(set)"` flag — never the value.
3. **Additive-only, verified-first.** The agent reads the endpoint's source first to confirm the operation is *additive* (the rotatable key lives at KV `admin_key:rotatable`, alongside — never replacing — the root `ADMIN_KEY`), so a failed or interrupted ceremony can never orphan the root credential.

9.2 The ceremony flow





9.3 The trust boundary (who-can-touch-what)

Every arrow that **creates** a secret is executed by the Human; every arrow that **checks** one is executed by the Agent. The root key never leaves the Cloudflare secret store — rotation happens one rung down (`admin_key:rotatable`), which is why a botched ceremony can't lock anyone out.

```
  ROOT  ── Cloudflare secret ──────────────  CLI only, Security Human, never moves
   │      (ADMIN_KEY)
   │  POST /admin/rotate-key  (Human runs interactively; Agent blocked)
   ▼
ROTATABLE ── KV: admin_key:rotatable ─────  delegate / agency, self-rotates,
   │                                         root-revocable (DELETE)
   │  POST /admin/provision-token
   ▼
TENANT (crm_t_*) ── KV per-shop ──────────  client B, shop-scoped, no admin access


  AGENT scope ▒▒▒▒  verify + record only — never mints, never holds a value
```

9.4 Audit record format

Every ceremony files one append-only record. It carries **fingerprints, never secrets** (`sha256(value)[:8]`), so the trail is itself safe to publish:

FIELD	EXAMPLE	NOTES
`date`	`2026-06-11T16:49:00Z`	UTC, ceremony completion
`credential`	`admin_key:rotatable`	logical name, not value
`action`	`rotate`	mint | rotate | revoke | set
`executed_by`	`security-human:ysl`	who ran the interactive command
`verified_by`	`agent:claude-code`	who probed old-dead / new-alive
`old_fp` → `new_fp`	`1a2b3c4d` → `564bbe92`	sha256[:8]; proves change without exposure
`reason`	`quarterly-cadence`	handoff | 90d | personnel | incident | scope
`overlap_window`	`0s (instant cutover)`	how long both keys were valid
`consumers_updated`	`worker secret; Actions secret`	every surface that had to change

9.5 Operator rules (reaffirmed)

- Secrets only via `$(cat ~/.file)` — never inline, never printed, never committed.
- A classifier-blocked privileged command is **handed to the operator** (run interactively), not worked around or retried with the guard disabled.
- Source is read **before** any retry, to confirm the operation is additive and the root credential is untouched.
- Local key inventory lives in `chmod 600` dotfiles (`~/.crm-admin-key` root, `~/.crm-agency-key` delegate, per-market site tokens) — never in the repo, never in chat.
- Client (B) gets **tenant token only** — no admin access of any kind

9.6 Challenge & solution — `.env` files and the CLI as an AI exposure surface

The conventional way to manage secrets across Cloudflare, Shopify, and Xano — a `.env` file on disk plus a `wrangler secret put` typed at the terminal — is *exactly* the surface that leaks to an AI agent. The ceremony exists because these two everyday conveniences become liabilities the moment a capable agent shares the workstation.

The challenge (three concrete exposure vectors):

- `.env` **is plaintext at rest.** Any agent with file-read or shell access can `cat` it. Once read, the secret lives in the model's context window *and* the chat transcript permanently — it cannot be "un-seen."
- **CLI arguments and stdin are capturable.** A secret passed inline (`wrangler secret put KEY <value>`, or a `curl -H "Authorization: Bearer sk_live_…"`) lands in shell history and in the tool-call log the agent can read back.
- **Echo-back on "verify."** The lazy way to confirm a key is to print it — which dumps the live value straight into context. Verification, not just storage, is an exposure point.

The solution (route every secret around the agent):

VECTOR	MITIGATION IN THIS STACK
`.env` plaintext	Privileged secrets are **never** kept in a checked-out `.env`. Worker secrets live as **Cloudflare secrets** (encrypted, *write-only* — settable but never readable back); tenant keys live in **KV**; local custody is `chmod 600` dotfiles (`~/.crm-admin-key`) read only by the human. Dev uses `.dev.vars` with throwaway keys (§3).
Inline CLI / history	The **human** runs the privileged command interactively; secrets are injected by reference — `$(cat ~/.crm-admin-key)` — never typed inline, never printed, never committed. A permission classifier **blocks** the agent from executing credential-minting commands and hands them to the operator.
Echo-back on verify	Read-back surfaces return a **masked preview** (`564bbe…9298`) or a `"(set)"` flag — never the value. The agent verifies by *side-effect*: a `200` on a real endpoint, an old key now returning `401`, a rotation-count delta, a fingerprint diff (§9.4).
Config-layer overrides	Rotation happens in **KV/tenant config** (which wins over `env`), so a key change needs no `.env` edit and no redeploy — the value never passes through a build artifact or env file.

Net posture: the `.env` file and the CLI are treated as **hostile-to-AI by default**. The human is the only party that ever handles a secret's value; the agent is the only party that keeps the (value-free, fingerprint-only) audit trail. Usefulness of the agent is preserved; custody of the secret is structurally withheld from it.

9.7 Ceremony roles in an organization (RACI)

The two-role model (§9.2) is a *minimum*: one Security Human + one Agent. In a staffed organization those duties distribute across named functional roles. The table below maps each to the ceremony — and which existing actor in §9.2 it embodies.

Two persona dimensions apply:

- **Organizational persona** — an internal functional role on the delivering team (below). These are *operators and approvers* of the ceremony.
- **Stakeholder persona** — the external tier that *receives* the result: **A** (App Creator), **B** (Shared / tenant-token only), **C** (Private Worker), defined in §5 /

§7. Organizational personas act *on behalf of* a stakeholder tier (typically A or C, since B never touches admin credentials).

ORGANIZATIONAL PERSONA	ORG MANDATE	ROLE IN THE CEREMONY	ARTIFACT OWNED	MAPS TO §9.2 ACTOR
Compliance Officer	Regulatory adherence (PCI-DSS Req. 3 key mgmt, SOC 2 CC6.1)	Attests rotation **cadence** is met; reviews the audit log (§9.4); signs incident-closure after a compromise (§8.2)	Rotation schedule (§8.1) + audit-record sign-off	Oversight — neither executes nor holds; consumes the fingerprint trail
DPO (Data Protection Officer)	GDPR Art. 37–39; protects the personal-data / PII plane	**Approves** any ceremony whose blast radius includes personal data — `JWT_SECRET` (sessions) and `XANO_API_KEY` (PII data store) — before it runs	Data-protection sign-off / DPIA note	Approval gate *upstream* of the Security Human
PMO (Project Mgmt Office)	Coordinates handoffs and cadence as scheduled events	**Initiates** the ceremony ("rotate the key"), owns this RACI, tracks the 90-day / handoff / personnel triggers	Ceremony calendar + handoff runbook (`AGENCY-HANDOFF.md`)	Coordinator — the request that opens step 1 of §9.2
QA / Release Manager	Release quality gate	**Verifies** the change on Stage before Prod; issues go/no-go that a rotation didn't break a release; co-signs the Agent's verification (§9.2 step 9)	Stage verification result + go/no-go record	Verifier — pairs with the Agent's probes
Deployment Officer	Executes privileged writes against the live environment	**Runs the interactive command** — `wrangler secret put` , `curl …/admin/rotate-key` — with the secret injected by reference; holds `chmod 600` dotfile custody	The executed command + new secret on disk	**Security Human (executor)** — the only role that touches a value

> **The Agent (AI/automation) maps to *none* of the human roles.** It prepares the runbook, runs the verification probes on the QA/Compliance officers' behalf, and files the fingerprint-only audit record — but never executes the privileged write and never holds a value (§9.1).

9.7.1 SEGREGATION OF DUTIES (THE POINT OF DUAL CONTROL)

The compliance value only holds if the roles **do not collapse into one person**:

- The **Deployment Officer** (who executes) must **not** also be the **Compliance Officer** (who attests) — self-attestation defeats the audit. PCI-DSS dual control is precisely this separation.
- The **DPO** approves *before* execution; the **Compliance Officer** reviews *after* — approval and attestation are separate gates, held by separate people.
- In a **small team**, one person may wear several hats, but the **executor and the attester must remain distinct**, and the **Agent always stays in the verify-and-record lane** regardless of headcount. Documenting which human held which hat per ceremony is itself part of the audit record (`executed_by` vs `verified_by`, §9.4).

---

10. Formal Definitions (Regulated-Industry Glossary)

These are the normative, citable definitions a regulated deployment (PCI-DSS, SOC 2, GDPR, ISO 27001, FIPS-validated environments) will expect when auditing this lifecycle. Where this project's terms are colloquial (e.g. "blast radius"), the formal equivalent is given so the control maps cleanly to an assessor's framework.

TERM (AS USED HERE)	FORMAL DEFINITION	AUTHORITATIVE SOURCE
Key ceremony	A formally scripted, witnessed, and audited procedure for executing a sensitive key-management operation (generation, rotation, revocation, distribution) under predefined controls and role separation.	NIST SP 800-57 Pt 2 (key-mgmt org); ICANN DNSSEC DPS; PCI PIN Security
Dual control	A process requiring **two or more** separate, authorized individuals to act together to perform a single operation, such that no one individual can perform it alone.	PCI-DSS Glossary; PCI PIN Security Req.
Split knowledge	A condition in which two or more parties separately hold key components that, individually, convey **no** knowledge of the resultant cryptographic key.	PCI-DSS Glossary; NIST SP 800-57 Pt 1
Separation / Segregation of Duties (SoD)	A control dividing a sensitive task across multiple actors so no single actor can both **execute and conceal** an action; reduces fraud and undetected error.	NIST SP 800-53 **AC-5**; ISO/IEC 27001:2022 A.5.3; SOC 2 CC
Least privilege	Granting each actor (human, service, or token) only the minimum access required to perform its function.	NIST SP 800-53 **AC-6**
Key custodian	An individual formally entrusted with, and accountable for protecting, a cryptographic key or key component (typically with a signed custodian acknowledgment).	PCI-DSS Req. 3 (key management)
Crypto Officer (CO)	A defined operator **role** authorized to perform initialization and key-management functions on a cryptographic module. (Distinct from the "User" role.)	FIPS 140-3 / ISO/IEC 19790
Cryptoperiod	The bounded time span during which a specific key is authorized for use; expiry mandates rotation.	NIST SP 800-57 Pt 1
Key rotation	Retiring an active key at the end of its cryptoperiod (or on compromise) and replacing it with a new key, without loss of service.	NIST SP 800-57 Pt 1
Scope of compromise *(our "blast radius")*	The complete set of resources, data, and operations reachable by a single credential if disclosed — the unit of risk that least-privilege and rotation bound.	NIST SP 800-53 (impact/containment)
Key fingerprint	A truncated cryptographic hash of a key used to **identify** it (in logs/audits) without revealing the key material. Here: `sha256(value)[:8]` (§9.4).	NIST SP 800-57 (key identifiers)

TERM (AS USED HERE)	FORMAL DEFINITION	AUTHORITATIVE SOURCE
Non-repudiation / audit trail	A tamper-evident, attributable record sufficient to prove who performed an action and that it occurred.	SOC 2 CC7; GDPR Art. 30 (records of processing)
RACI	A responsibility-assignment matrix classifying each actor against an activity as **R**esponsible (does the work), **A**ccountable (the *single* owner who answers for the outcome — exactly one per activity), **C**onsulted (two-way input before action), or **I**nformed (one-way notification after). Variants: **RASCI** (+ **S**upport), **RACI-VS** (+ **V**erifies / **S**igns-off), **CAIRO/RACIO** (+ **O**mitted/out-of-loop).	PMI *PMBOK Guide*; ISO 21500
Data Protection Officer (DPO)	A mandated independent role responsible for monitoring an organization's personal-data protection compliance. **Must not** also *determine* the purposes/means of processing (conflict-of-interest bar) — hence advisory, never Accountable.	GDPR **Art. 37–39**
Security of processing	The obligation to implement appropriate technical measures — incl. encryption and the ability to ensure ongoing confidentiality — proportionate to risk.	GDPR **Art. 32**
Write-only secret	A credential surface that accepts a value on set but **never returns it** on read (read-back yields a mask or `(set)` flag); prevents echo-back disclosure.	Project control (§9.6); aligns NIST SP 800-57 protection-of-secrets

> **Mapping note for assessors.** §9 (Interactive Key Ceremony) is the *implementation* of **dual control + SoD + least privilege**, extended so that an **AI/automation agent is treated as a non-custodial actor** — it may verify and record but never satisfies the "authorized individual" requirement for the privileged write. §9.4 (audit record) satisfies non-repudiation; §8.1 (rotation schedule) satisfies cryptoperiod bounds.

11. Significance in Recent Shopify & Google GraphQL / Server-Side Migrations

The two platform migrations this stack rides on — **Shopify REST → GraphQL Admin** and **Google UA → GA4 (server-side measurement)** — are not just API churn. Both move credential-bearing work from *many low-privilege, client-exposed identifiers* to *fewer, high-privilege, server-side secrets concentrated in the edge worker*. That concentration is exactly what makes the key lifecycle in this document load-bearing rather than optional. See `SHOPIFY-2026-RISK-BRIEF.md` and `FEATURE-SPEC-UA-MIGRATION.md`.

11.1 Shopify: REST → GraphQL Admin API

- **One endpoint, one token, broad surface.** The GraphQL Admin API exposes a single endpoint (`/admin/api/<version>/graphql.json`) guarded by one Admin access token. The token's **scope of compromise (§10)** is now the *entire granted schema* — queries and mutations alike — not a single REST route. → **Control:** least-privilege scoping (request only the `read_*` / `write_*` scopes actually used) and disciplined rotation become the primary defenses, not an afterthought.
- **Platform-native token rotation.** Shopify Admin tokens are increasingly short-lived / expiring, and the worker **auto-rotates** them (§2, `SHOPIFY_ADMIN_TOKEN`). The platform's own direction now *matches* the cryptoperiod discipline of §8.1 — rotation is no longer a manual chore the org might skip.
- **Server-side is mandatory, not optional.** A GraphQL Admin token must never reach the browser; all calls run server-side in the Cloudflare worker. The worker becomes the **credential boundary** — which is precisely why the `.env`/CLI-as-AI-exposure threat model (§9.6) applies to the live system, and why the ceremony keeps the agent that *generates GraphQL code* out of secret custody.

11.2 Google: Universal Analytics → GA4 (server-side measurement)

- **A real secret appears where there wasn't one.** Client-side UA/gtag used only a public measurement ID (no secret). GA4 **server-side measurement** (Measurement Protocol / server-side tagging) requires a genuine `api_secret` (`GA4_API_SECRET`, §2). Measurement moving server-side moves the secret **into the worker** — a new custodial asset that must be set write-only and rotated.
- **Consent-gated credentials.** GA4 + **Consent Mode v2** (required in the EEA for ad-personalization signals) couples credential use to a lawful-basis gate — bringing the **DPO (§9.7, GDPR Art. 37–39)** and **security-of-processing (Art. 32, §10)** into the key lifecycle, not just analytics config.
- **Higher-trust data paths.** GA4 Data API / BigQuery export use OAuth / service-account credentials whose scope of compromise includes historical analytics data — again favoring least-privilege scoping over broad, long-lived grants.

11.3 The common thread (why this document matters now)

BEFORE (LEGACY)	AFTER (GRAPHQL / SERVER-SIDE)	CONSEQUENCE FOR KEY MGMT
Many REST routes, scoped per call	One GraphQL endpoint, one token	Larger **scope of compromise** per credential → least-privilege + rotation are primary controls
Public client-side IDs (gtag, Storefront)	Server-side secrets in the worker	Secrets **concentrate** at the edge → §9.6 `.env` /CLI threat model is live, not theoretical
Static, long-lived tokens	Expiring / rotating tokens	Cryptoperiod (§10) is enforced **by the platform** → §8.1 cadence aligns with vendor direction
Humans wrote integrations	**AI agents** generate GraphQL/codegen + run server-side ops	The automation building the integration must be **non-custodial** → §9.1 / §9.7 keep it in the verify-and-record lane

Bottom line: GraphQL's single-endpoint model and Google's server-side measurement both *concentrate* credential power into fewer, server-resident secrets — at the same moment AI agents are doing more of the integration

work. That intersection is the reason the **Interactive Key Ceremony** (formal dual control + SoD, extended to treat the agent as non-custodial) is the right — and arguably required — control for a regulated deployment of this stack.

---

12. Ownership — Who Owns This, From Tech to Business

The **RACI** model (defined in §10) requires that every activity have **exactly one Accountable owner**. This section names that owner end-to-end and shows how ownership escalates from the technical floor to the business.

Legend: R = Responsible (does it) · **A** = Accountable (owns the outcome, one only) · **C** = Consulted (input before) · **I** = Informed (told after).

12.1 The single point of accountability

- **Business owner / Accountable for the lifecycle as a whole: the CISO** (or, in a smaller org, the **Head of Security / VP Engineering** who carries the risk mandate). Accountability for *risk* is a business function — it cannot sit with an individual contributor or an outside consultant.
- **Technical Accountable (delegated): the Engineering Lead** for the mechanism itself — the worker, the KV/secret stores, the rotation endpoints. The CISO owns *that the control works*; the Engineering Lead owns *how it works*.
- **The DPO is Consulted, never Accountable.** GDPR's conflict-of-interest bar (§10) forbids the DPO from owning the processing decision; they advise and monitor.
- **The Agent (AI/automation) never appears as A or R for a privileged write** — only as a support actor for verification/recording (§9.1).

12.2 RACI across the lifecycle (tech → business)

ACTIVITY	DEPLOYMENT OFFICER	ENGINEERING LEAD	QA / RELEASE MGR	PMO	COMPLIANCE OFFI
Build / maintain the mechanism (worker, rotate endpoints)	C	**A/R**	C	I	I
Schedule a ceremony (cadence / handoff / incident trigger)	I	C	C	**A/R**	C
Approve a ceremony touching the PII plane (`JWT_SECRET`, `XANO_API_KEY`)	I	C	I	I	C
Execute the privileged write (mint / rotate / revoke / set)	**A/R**	C	I	I	I
Verify old-dead / new-alive (non-secret probes)	C	C	**A/R** *(with Agent support)*	I	I
Stage-before-prod go / no-go	I	C	**A/R**	C	I
Review audit record + attest cadence met	I	I	C	I	**A/R**
Incident response on key compromise (§8.2)	R	R	C	C	**R**

ACTIVITY	DEPLOYMENT OFFICER	ENGINEERING LEAD	QA / RELEASE MGR	PMO	COMPLIANCE OFFI
Own residual risk / sign the control as effective	I	C	I	I	C

> **Reading the chain:** technical *Responsibility* concentrates on the **Deployment Officer** (execution) and **Engineering Lead** (mechanism); operational *coordination* on the **PMO**; assurance on **QA** and **Compliance**; lawful-basis advice on the **DPO**; and ultimate *Accountability for risk* on the **CISO / business**. Every row has exactly one **A** — the model's core invariant.

12.3 Smaller-org collapse (and the lines that must not collapse)

In a lean team one person may hold several letters, **except** the two invariants from §9.7.1: the **executor (Deployment Officer) and the attester (Compliance Officer) must stay distinct**, and **Accountability (A) must sit with a business risk owner, not the person doing the work (R)**. If a single founder wears every hat, the **Agent's non-custodial verify-and-record role becomes the de-facto second control** — which is why the ceremony is designed to hold even at headcount = 1.

---

13. Self-Service Tenant Key Management (App Owners)

*Added v1.5 (2026-07-03). The buyer-facing tier of the key hierarchy: every app purchase includes a dedicated per-store admin credential that the **owner** — not the platform — mints, rotates, revokes, and audits.*

13.1 Position in the key hierarchy

TIER	CREDENTIAL	HELD BY	RESET BLAST RADIUS
Platform root	worker root admin key	Platform operator only	Everything (full dependent rotation)
Delegate	rotatable admin key	Platform / consulting ops	The delegate only
App owner	**per-store tenant token**	**The purchasing owner only**	**That one store only**

The platform root and delegate tiers are covered by §§3–9. This section covers the third tier, which is the one sold with the product: the owner's dedicated key.

13.2 The owner lifecycle (wizard)

Owners manage their key through a guided wizard — sign in, confirm the store, generate. The design constraints mirror the ceremony's invariants, translated to self-service:

1. **Entitlement-gated ownership.** A signed-in identity may manage a store's keys only if its entitlement record — written by **purchase**, not self-serviceable — lists that store. A login alone never escalates to key access.
2. **Shown once.** The key value is returned exactly once at mint. Every subsequent view is a truncated fingerprint (`sha256[:8]`) and a masked preview. Losing the key means rotating it — by design, there is nothing to "look up."
3. **Mint-before-revoke rotation.** The replacement key is written before predecessors are deleted, so no failure mode leaves the store keyless (the self-service analogue of §9's additive-only rule).
4. **Incident revoke.** Any single key can be killed by fingerprint without a replacement — the owner's kill switch for a suspected leak.
5. **Append-only audit.** Every rotate/revoke appends a fingerprint-only record (date, action, actor, old → new fingerprints) to a per-store ledger the owner can read in the wizard — the same record shape a ceremony files (§9.5), produced automatically.

13.3 Session integrity underneath the wizard

The wizard is only as trustworthy as the login in front of it:

- **Server-side logout revocation.** Logging out denylists the session token at the server (hash, TTL = remaining token lifetime). A cached copy of the session — browser storage, history, a shared machine — dies with the logout instead of surviving to its natural expiry.
- **Click-time token passing.** Authenticated surfaces receive the session token at interaction time and hold it in memory; it is scrubbed from URLs and never persisted by the receiving page.

13.4 Why this is the product, not a feature

A SaaS contract asks the customer to trust the vendor's custody of credentials and a document's promise about it. This model inverts custody: the platform operates the key *system* — mint, rotation, revocation, audit — while the customer holds the key *value*. The operator cannot leak what it never held; the owner cannot be locked out of what only they possess. Peer teams extend the same property to their agents: agents act under scoped, revocable mandates and never hold the owner's key (§9.1's non-custodial rule, applied to the customer's own automation).
